University of Science and Technology Chittagong (USTC)

Department of Computer Science and Engineering



Lab Report-3

Course Title: Computer Graphics and Image Processing Lab

Course Code: CSE 418

Submitted To

Umme Aiman Jannat
Lecturer
CSE, FSET, USTC

Submitted By

Name: **Bijon Chowdhury Prem**
Roll No: **23070165**
Registration No: **1071**
Batch: **41 (Section-B, CSE)**
7th Semester

Experiment – 1

Source Code: (Image uploading)

```
[1]
✓ 39s
```

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload Goku image
uploaded = files.upload()

filename = list(uploaded.keys())[0]

# Read image
img = cv2.imread(filename)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

print("Image loaded successfully!")
print("Image dimensions:", img.shape)

# Display original image
plt.figure(figsize=(5, 7))
plt.imshow(img)
plt.title("Original Goku Image")
plt.axis("off")
plt.show()
```

Output:

Choose Files goku image.jpeg

goku image.jpeg(image/jpeg) - 28132 bytes, last modified: 11/7/2026 - 100% done

```
Saving goku image.jpeg to goku image.jpeg
Image loaded successfully!
Image dimensions: (597, 335, 3)
```



Source Code:

[2] ✓ 0s

```
# EXPERIMENT 1
# Histogram Calculation and Histogram Equalization

# Calculate histogram
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])

# Apply histogram equalization
equalized = cv2.equalizeHist(gray)

# Calculate equalized histogram
hist_equalized = cv2.calcHist(
    [equalized], [0], None, [256], [0, 256]
)

# Display outputs
plt.figure(figsize=(14, 9))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Goku Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.plot(hist)
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.subplot(2, 2, 3)
plt.imshow(equalized, cmap='gray')
plt.title("Histogram Equalized Image")
plt.axis("off")

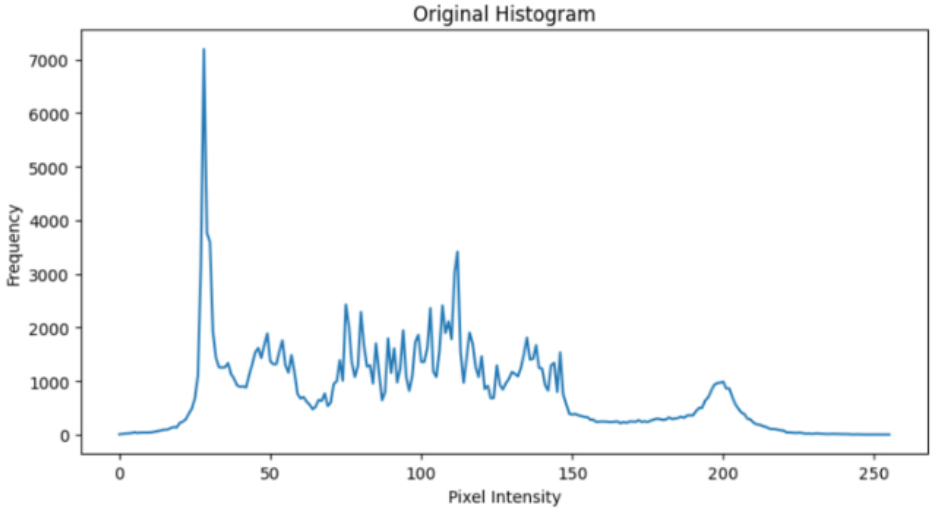
plt.subplot(2, 2, 4)
plt.plot(hist_equalized)
plt.title("Equalized Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Output:

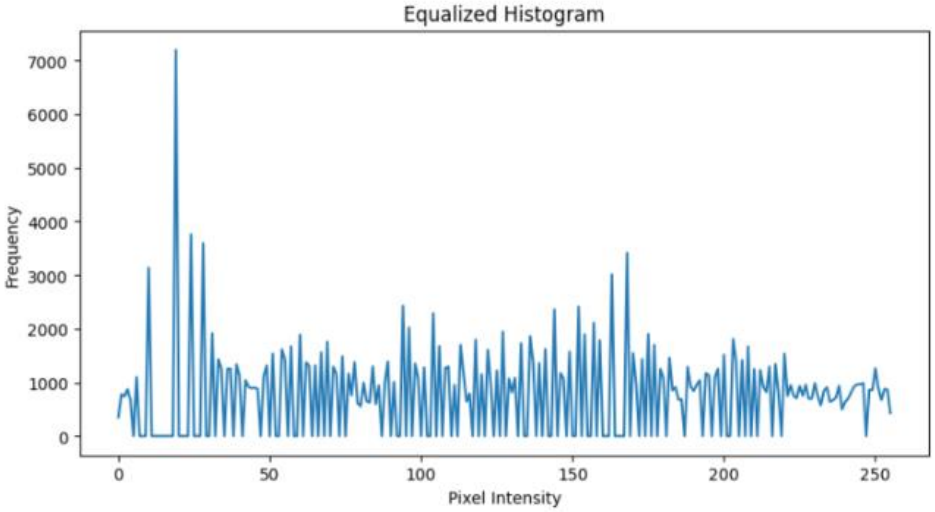
Original Goku Image





Histogram Equalized Image





Experiment – 2

Source Code:

[3] ✓ 2s

```
# EXPERIMENT 2
# Logical Operations: AND, OR and NOT

# Create binary mask
_, mask = cv2.threshold(
    gray, 127, 255, cv2.THRESH_BINARY
)

# Logical AND
and_result = cv2.bitwise_and(gray, mask)

# Logical OR
or_result = cv2.bitwise_or(gray, mask)

# Logical NOT
not_result = cv2.bitwise_not(gray)

# Display outputs
plt.figure(figsize=(14, 9))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Goku Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(and_result, cmap='gray')
plt.title("AND Operation")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(or_result, cmap='gray')
plt.title("OR Operation")
plt.axis("off")
```

```
plt.subplot(2, 2, 4)
plt.imshow(not_result, cmap='gray')
plt.title("NOT Operation")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:

Original Goku Image



AND Operation



OR Operation



NOT Operation



Experiment – 3

Source Code:

[5]
✓ 0s

```
# EXPERIMENT 3
# Arithmetic Operations

# Create constant image
constant = np.full(gray.shape, 100, dtype=np.uint8)

# Addition
addition = cv2.add(gray, constant)

# Subtraction
subtraction = cv2.subtract(gray, constant)

# Multiplication
multiplication = cv2.multiply(
    gray.astype(np.float32),
    np.full(gray.shape, 1.5, dtype=np.float32)
).astype(np.uint8)

# Division
division = cv2.divide(
    gray.astype(np.float32),
    np.full(gray.shape, 2.0, dtype=np.float32)
).astype(np.uint8)

# Display outputs
plt.figure(figsize=(14, 10))

plt.subplot(2, 3, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Goku Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(addition, cmap='gray')
plt.title("Addition")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(subtraction, cmap='gray')
plt.title("Subtraction")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(multiplication, cmap='gray')
plt.title("Multiplication")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(division, cmap='gray')
plt.title("Division")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:

Original Goku Image



Addition



Subtraction



Multiplication



Division



Experiment – 4

Source Code:

[6] ✓ 1s

```
# EXPERIMENT 4
# Image Filtering

# 1. Sobel Operator
sobel_x = cv2.Sobel(
    gray, cv2.CV_64F, 1, 0, ksize=3
)
sobel_y = cv2.Sobel(
    gray, cv2.CV_64F, 0, 1, ksize=3
)

sobel = cv2.magnitude(sobel_x, sobel_y)
sobel = cv2.convertScaleAbs(sobel)

# 2. Box Filter
box_filter = cv2.boxFilter(
    gray, -1, (3, 3)
)

# 3. Weighted Average Filter
weighted_kernel = np.array([
    [1, 2, 1],
    [2, 4, 2],
    [1, 2, 1]
], dtype=np.float32) / 16

weighted_filter = cv2.filter2D(
    gray, -1, weighted_kernel
)

# 4. Gaussian Filter
gaussian = cv2.GaussianBlur(
    gray, (5, 5), 0
)

# 5. Sharpening Filter
sharpen_kernel = np.array([
    [0, -1, 0],
    [-1, 5, -1],
    [0, -1, 0]
])

sharpen = cv2.filter2D(
    gray, -1, sharpen_kernel
)

# 6. Minimum Filter
minimum = cv2.erode(
    gray, np.ones((3, 3), np.uint8)
)

# 7. Maximum Filter
maximum = cv2.dilate(
    gray, np.ones((3, 3), np.uint8)
)

# 8. Mean Filter
mean_filter = cv2.blur(
    gray, (3, 3)
)

# 9. Median Filter
median = cv2.medianBlur(
    gray, 3
)

# Display all outputs
images = [
    ("Original Goku Image", gray),
    ("Sobel Operator", sobel),
    ("Box Filter", box_filter),
    ("Weighted Average", weighted_filter),
    ("Gaussian Filter", gaussian),
    ("Sharpening Filter", sharpen),
    ("Minimum Filter", minimum),
    ("Maximum Filter", maximum),
    ("Mean Filter", mean_filter),
    ("Median Filter", median)
]

plt.figure(figsize=(15, 17))

for i, (title, result) in enumerate(images):
    plt.subplot(4, 3, i + 1)
    plt.imshow(result, cmap='gray')
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:

Original Goku Image



Sobel Operator



Box Filter



Minimum Filter



Maximum Filter



Mean Filter



Weighted Average



Gaussian Filter



Sharpening Filter



Median Filter



Experiment – 5

Source Code:

```
# EXPERIMENT 5
# Laplacian Operations

# 1. Laplacian Mask
laplacian = cv2.Laplacian(
    gray, cv2.CV_64F, ksize=3
)

laplacian = cv2.convertScaleAbs(laplacian)

# 2. Enhanced Laplacian
laplacian_signed = cv2.Laplacian(
    gray, cv2.CV_32F, ksize=3
)

# Sharpen image using Laplacian
enhanced_laplacian = cv2.subtract(
    gray,
    cv2.convertScaleAbs(laplacian_signed)
)

# 3. High-Boost Filtering
blurred = cv2.GaussianBlur(
    gray, (5, 5), 0
)

# Create high-boost mask
high_boost_mask = cv2.subtract(
    gray, blurred
)

# High-boost output
high_boost = cv2.addWeighted(
    gray, 2.0, blurred, -1.0, 0
)

# Display outputs
plt.figure(figsize=(14, 10))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Goku Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(laplacian, cmap='gray')
plt.title("Laplacian Mask")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(enhanced_laplacian, cmap='gray')
plt.title("Enhanced Laplacian")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(high_boost, cmap='gray')
plt.title("High-Boost Filtering")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:

Original Goku Image



Laplacian Mask



Enhanced Laplacian



High-Boost Filtering

